

	0614-Desplegament d'Aplicacions Web		
	RA4: Administra servidors de transferència de fitxers		
Garrido Rubio, Pablo			
Práctica N.º:	1	Título práctica:	Microservicio Facial FastApi / Deepface

Microservicio Facial FastApi / Deepface

1.Introducción

El objetivo de esta práctica es implementar un sistema de reconocimiento facial utilizando una arquitectura de microservicios. Se integra un backend en **Laravel** con un servicio especializado en IA desarrollado en **Python (FastAPI)**, todo orquestado mediante **Docker** en un entorno de Raspberry Pi.

2.Configuración del Entorno Servidor

1.1. Ajustes de PHP (php.ini)

PHP bloquea por defecto la subida de archivos de más de 2 MB. Para que las fotos de la webcam puedan subirse correctamente, hemos modificado el archivo php.ini con los siguientes valores:

- **Cambios realizados:**
 - upload_max_filesize: 20M

```
; Maximum allowed size for uploaded files.
; https://php.net/upload-max-filesize
upload_max_filesize = 20M
```

Pablo Garrido

- post_max_size: 25M

```
; Maximum size of POST data that PHP will accept.
; Its value may be 0 to disable the limit. It is ignored if POST data reading
; is disabled through enable_post_data_reading.
; https://php.net/post-max-size
post_max_size = 25M
```

Pablo Garrido

3. Infraestructura y Despliegue (Docker)

Se ha utilizado **Docker Compose** para aislar el microservicio de IA y la base de datos, evitando conflictos de puertos con servicios locales.

- **Servicio Facial:** Expuesto en el puerto 8181 (mapeado al 5000 interno).
- **Base de Datos:** PostgreSQL configurado en el puerto 5433 para evitar colisiones con el puerto por defecto (5432).

```
docker-compose.yml X
docker-compose.yml
1  version: '3.8'
   Run All Services
2  services:
   Run Service
3    facial-api:
4      build: ./microservicio_facial
5      container_name: facial_service
6      ports:
7        - "8181:5000"
8      restart: always
   Run Service
9    db:
10     image: postgres:15-alpine
11     container_name: postgres_db
12     ports:
13       - "5433:5432"
14     environment:
15       POSTGRES_DB: plataforma_juegos
16       POSTGRES_USER: admin
17       POSTGRES_PASSWORD: secretpassword
18     volumes:
19       - pgdata:/var/lib/postgresql/data
20 volumes:
21   pgdata:
```

Pablo Garrido

4. Desarrollo del Microservicio de IA (Python)

El archivo main.py recibe las imágenes, las guarda temporalmente y llama a DeepFace para verificar si las dos caras pertenecen a la misma persona.

```
main.py X
microservicio_facial > main.py
1  from fastapi import FastAPI, File, UploadFile
2  from deepface import DeepFace
3  import shutil
4  import os
5
6  app = FastAPI()
7
8  @app.post("/verify")
9  async def verify_img(img1: UploadFile = File(...), img2: UploadFile = File(...)):
10     path1 = f"temp_{img1.filename}"
11     path2 = f"temp_{img2.filename}"
12
13     try:
14
15         with open(path1, "wb") as buffer:
16             shutil.copyfileobj(img1.file, buffer)
17         with open(path2, "wb") as buffer:
18             shutil.copyfileobj(img2.file, buffer)
19
20
21         result = DeepFace.verify(
22             img1_path=path1,
23             img2_path=path2,
24             model_name="Facenet",
25             detector_backend="opencv",
26             enforce_detection=False
27         )
28
29         return {
30             "verified": bool(result["verified"]),
31             "distance": float(result["distance"]),
32             "model": result["model"]
33         }
34     finally:
35         if os.path.exists(path1): os.remove(path1)
36         if os.path.exists(path2): os.remove(path2)
```

5. Frontend y Captura WebRTC

Integración de WebRTC mediante JavaScript. Se captura el flujo de vídeo, se congela un fotograma usando un elemento y se inyecta como blob.

```
@if(isset($resultado))
    <div class="panel-resultado">
        <h3>Resultado de la Inteligencia Artificial:</h3>
        @if(isset($resultado['verified']) && $resultado['verified'])
            <div class="success">✅ ¡Identidad Verificada Correctamente!</div>
        @else
            <div class="error">❌ Identidad NO verificada (No coinciden).</div>
        @endif

        <p><strong>Nivel de diferencia (Distancia):</strong> {{ $resultado['distance'] ?? 'N/A' }}</p>
        <p><strong>Modelo usado:</strong> {{ $resultado['model'] ?? 'N/A' }}</p>
    </div>
@endif

<video id="video" width="640" height="480" autoplay></video>
<canvas id="canvas" width="640" height="480" style="display:none;"></canvas>

<form id="formulario-facial" method="POST" action="/test-facial" enctype="multipart/form-data">
    @csrf
    <div style="margin-bottom: 15px;">
        <label><strong>1. Sube tu foto de registro (DNI/Perfil):</strong></label><br>
        <input type="file" name="foto_registro" required>
    </div>

    <input type="file" id="foto_webcam" name="foto_webcam" style="display:none;">

    <button type="button" id="btn-capturar" style="padding: 10px 20px; font-size: 1.1em; cursor: pointer;">
        Capturar y Verificar
    </button>
</form>
```

Pablo Garrido

7. Despliegue de la Aplicación Laravel en Raspberry Pi

Una vez que los contenedores Docker (base de datos e IA) estaban en marcha, el siguiente paso fue preparar y arrancar el proyecto Laravel directamente en la Raspberry Pi para que se conectara a ellos.

Instalar PHP y Composer

Laravel necesita PHP y Composer para funcionar. Se instalaron junto con el driver de PostgreSQL ejecutando el siguiente comando:

```
sudo apt update
```

```
sudo apt install php-cli php-mbstring php-xml php-pgsql unzip curl git -y
```

```
pablo@pablopi:/var/www/reconocimiento_facial/aplicacion-facial $ sudo apt update
Hit:1 http://deb.debian.org/debian trixie InRelease
Hit:2 http://deb.debian.org/debian trixie-updates InRelease
Hit:3 http://deb.debian.org/debian-security trixie-security InRelease
Hit:4 http://archive.raspberrypi.com/debian trixie InRelease
19 packages can be upgraded. Run 'apt list --upgradable' to see them.
pablo@pablopi:/var/www/reconocimiento_facial/aplicacion-facial $ sudo apt install php-cli php-mbstring php-xml php-pgsql unzip curl git -y
php-mbstring is already the newest version (2:8.4+96).
php-xml is already the newest version (2:8.4+96).
php-pgsql is already the newest version (2:8.4+96).
unzip is already the newest version (6.0-29).
```

Pablo Garrido

Crear y configurar el archivo .env

Laravel requiere un archivo `.env` con los datos reales de conexión. Se creó mediante la plantilla de ejemplo y se editó con `nano` para sustituir la configuración de SQLite por la del contenedor PostgreSQL:

Los parámetros de base de datos establecidos en el archivo `.env` fueron los siguientes:

```
DB_CONNECTION=pgsql
DB_HOST=127.0.0.1
DB_PORT=5433
DB_DATABASE=plataforma_juegos
DB_USERNAME=admin
DB_PASSWORD=secretpassword
```

```
GNU nano 8.4
# APP_MAINTENANCE_STORE=database

# PHP_CLI_SERVER_WORKERS=4

BCRYPT_ROUNDS=12

LOG_CHANNEL=stack
LOG_STACK=single
LOG_DEPRECATED_CHANNELS=null
LOG_LEVEL=debug

DB_CONNECTION=pgsql
DB_HOST=127.0.0.1
DB_PORT=5433
DB_DATABASE=plataforma_juegos
DB_USERNAME=admin
DB_PASSWORD=secretpassword

SESSION_DRIVER=database
SESSION_LIFETIME=120
```

Pablo Garrido

Instalar las dependencias de Laravel

Con la configuración lista, se descargaron todas las dependencias internas del framework mediante Composer:

```
composer install
```

```
pablo@pabloi:/var/www/reconocimiento_facial/aplicacion-facial $ composer install
The repository at "/var/www/reconocimiento_facial/aplicacion-facial" does not have
git refuses to use it:

fatal: detected dubious ownership in repository at '/var/www/reconocimiento_facial'
To add an exception for this directory, call:

git config --global --add safe.directory /var/www/reconocimiento_facial

Installing dependencies from lock file (including require-dev)
Verifying lock file contents can be installed on current platform.
Package operations: 111 installs, 0 updates, 0 removals
- Downloading doctrine/inflector (2.1.0)
- Downloading doctrine/lexer (3.0.1)
- Downloading dragonmantank/cron-expression (v3.6.0)
- Downloading symfony/deprecation-contracts (v3.6.0)
- Downloading psr/container (2.0.2)
- Downloading fakerphp/faker (v1.24.1)
- Downloading symfony/polyfill-mbstring (v1.33.0)
```

Pablo Garrido

Generar la clave de seguridad

Laravel necesita una clave única para encriptar las sesiones de usuario. Se generó automáticamente con:

```
php artisan key:generate
```

```
pablo@pabloi:/var/www/reconocimiento_facial/aplicacion-facial $ php artisan key:generate
INFO Application key set successfully.
```

Pablo Garrido

Crear las tablas en la Base de Datos

Con Laravel ya conectado al contenedor PostgreSQL configurado en el Paso 2, se ejecutaron las migraciones para crear todas las tablas necesarias:

```
php artisan migrate
```

```
pablo@pablopi:/var/www/reconocimiento_facial/aplicacion-facial $ php artisan migrate

INFO  Preparing database.

Creating migration table ..... 51.32ms DONE

INFO  Running migrations.

0001_01_01_000000_create_users_table ..... 41.27ms DONE
0001_01_01_000001_create_cache_table ..... 26.21ms DONE
0001_01_01_000002_create_jobs_table ..... 38.04ms DONE
```

Pablo Garrido

Levantar la Web y el Túnel Ngrok

Para exponer la aplicación al exterior (necesario para que la cámara funcione vía HTTPS), se utilizaron dos terminales en paralelo.

Terminal 1 — Servidor Laravel: Desde la carpeta del proyecto, se arrancó el servidor integrado de PHP escuchando en todas las interfaces de red:

```
php artisan serve --host=0.0.0.0 --port=8000
```

```
pablo@pablopi:/var/www/reconocimiento_facial/aplicacion-facial $ php artisan serve --host=0.0.0.0 --port=8000

INFO  Server running on [http://0.0.0.0:8000].

Press Ctrl+C to stop the server

2026-04-09 18:41:43 / ..... ~ 500.37ms
2026-04-09 18:41:43 /favicon.ico ..... ~ 0.07ms
2026-04-09 18:43:11 /test-facial ..... ~ 11s
```

Pablo Garrido

Terminal 2 — Túnel Ngrok: Para obtener una URL pública HTTPS, se instaló Ngrok y se creó el túnel apuntando al puerto de Laravel:

```
curl -s https://ngrok-agent.s3.amazonaws.com/ngrok.asc | sudo tee
/etc/apt/trusted.gpg.d/ngrok.asc >/dev/null

echo "deb https://ngrok-agent.s3.amazonaws.com buster main" | sudo tee
/etc/apt/sources.list.d/ngrok.list

sudo apt update && sudo apt install ngrok
```

```
Last login: Thu Apr  9 18:18:39 2020 from 198.51.100.1:220%eth0
pablo@pablopi:~$ curl -s https://ngrok-agent.s3.amazonaws.com/ngrok.asc | sudo tee /etc/apt/trusted.gpg.d/ngrok.asc
>/dev/null
pablo@pablopi:~$ echo "deb https://ngrok-agent.s3.amazonaws.com buster main" | sudo tee /etc/apt/sources.list.d/ngro
ok.list
deb https://ngrok-agent.s3.amazonaws.com buster main
pablo@pablopi:~$ sudo apt update && sudo apt install ngrok
Hit:1 http://deb.debian.org/debian trixie InRelease
Hit:2 http://deb.debian.org/debian trixie-updates InRelease
Hit:3 http://deb.debian.org/debian-security trixie-security InRelease
Hit:4 http://archive.raspberrypi.com/debian trixie InRelease
Get:5 https://ngrok-agent.s3.amazonaws.com buster InRelease [20.3 kB]
Get:6 https://ngrok-agent.s3.amazonaws.com buster/main arm64 Packages [10.8 kB]
Get:7 https://ngrok-agent.s3.amazonaws.com buster/main armhf Packages [10.9 kB]
```

Pablo Garrido

Una vez instalado, he iniciado sesión en la página de Ngrok, para generar un token para poder arrancar el túnel, y guarde la configuración mediante este comando:

```
ngrok config add-authtoken 3C81o67roY6zV4287mRd2LF24rQ_5E6TgMgXyEqJutJPhmbqz
```

```
pablo@pablopi:~$ ngrok config add-authtoken 3C81o67roY6zV4287mRd2LF24rQ_5E6TgMgXyEqJutJPhmbqz
Authtoken saved to configuration file: /home/pablo/.config/ngrok/ngrok.yml
```

Pablo Garrido

se arrancó el túnel:

```
ngrok http 8000
```

```
pablo@pablopi:~$ ngrok http 8000
```

Pablo Garrido

Ngrok generó una URL pública con HTTPS que se abrió desde el navegador del ordenador con Windows 11. La aplicación cargó correctamente y solicitó permisos de cámara, confirmando el despliegue completo del sistema.


```
pablo@pablopi: ~
ngrok
One gateway for every AI model. Available in early access *now*: https://ngrok.com/r/ai

Session Status      online
Account             pablogarridok (Plan: Free)
Version             3.37.3
Region              Europe (eu)
Web Interface        http://127.0.0.1:4040
Forwarding            https://unglad-overfruitfully-caden.ngrok-free.dev -> http://localhost:8000

Connections          ttl    opn    rt1    rt5    p50    p90
                     0      0      0.00   0.00   0.00   0.00
```

Pablo Garrido

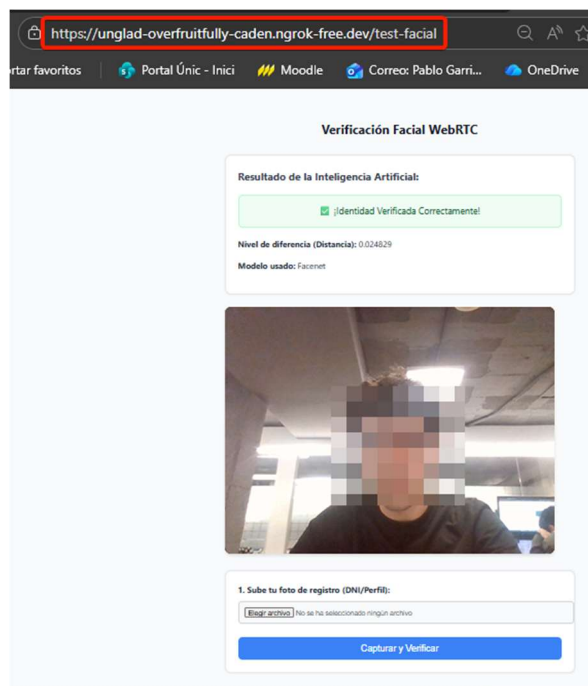
6. Pruebas y Resultados

Terminal mostrando los contenedores de Docker (la base de datos e inteligencia artificial) corriendo mediante el comando: **docker ps**:

```
pablo@pablopi:/var/www/reconocimiento_facial/aplicacion-facial $ docker ps
CONTAINER ID   IMAGE                                COMMAND                  CREATED        STATUS        PORTS                                                                 NAMES
6fdc3339aa54   aplicacion-facial-facial-api        "uvicorn main:app --..." 2 hours ago    Up About an hour    0.0.0.0:8181->5000/tcp, :::8181->5000/tcp    facial_service
da72897841d0   postgres:15-alpine                 "docker-entrypoint.s..." 2 hours ago    Up 46 minutes      0.0.0.0:5433->5432/tcp, :::5433->5432/tcp    postgres_db
```

Pablo Garrido

- **Validación correcta:**



Pablo Garrido

- **Validación incorrecta:**

https://unglad-overfruitfully-caden.ngrok-free.dev/test-facial

Portal Únic - Inici Moodle Correo: Pablo Garri... OneDrive

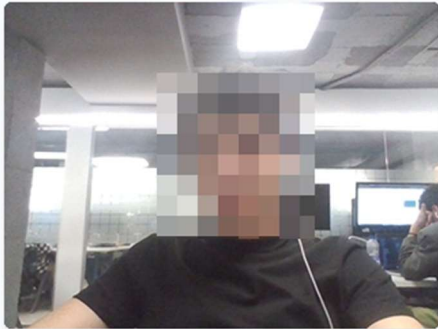
Verificación Facial WebRTC

Resultado de la Inteligencia Artificial:

✗ Identidad NO verificada (No coinciden).

Nivel de diferencia (Distancia): 0.659813

Modelo usado: Facenet



1. Sube tu foto de registro (DNI/Perfil):

No se ha seleccionado ningún archivo

Pablo Garrido